

Responsive Web Design and Introduction to JavaScript

Introduction: Building Websites for Every Device

In today's digital world, people access websites from a wide variety of devices — smartphones, tablets, laptops, and desktop computers. A website that looks great on a desktop screen might be impossible to use on a phone. This is where **responsive web design** comes in. Responsive design ensures that web pages adapt seamlessly to any screen size, providing an optimal viewing experience for every user.

CSS provides powerful tools for creating responsive layouts, including **Flexbox** for flexible arrangements and **Media Queries** for applying different styles based on screen size. Beyond layout, modern websites also need interactivity. This is where **JavaScript** enters the picture. JavaScript is a programming language that brings web pages to life by enabling dynamic content, user interactions, and complex functionality.

This reading material examines how to create responsive websites using CSS Flexbox, media queries, and RWD layouts. It also introduces the fundamentals of JavaScript, including its history, syntax, variables, data types, functions, and basic programming concepts like algorithms and flowcharts. By the end of this lesson, learners will be able to build websites that work on any device and understand the basics of programming logic.

Part 1: CSS Flexbox – Flexible Layouts

Introduction to Flexbox

Flexbox (Flexible Box Layout) is a CSS layout model that provides an efficient way to arrange items in a container, even when their sizes are unknown or dynamic. It is designed for one-dimensional layouts — either a row or a column.

The Flex Container

To use Flexbox, set the `display` property of the container to `flex`.

```
css
.container {
  display: flex;
}
```

Flexbox Properties

Property	Description
<code>flex-direction</code>	Defines the direction of the main axis (row, column)
<code>flex-wrap</code>	Controls whether items wrap to the next line
<code>justify-content</code>	Aligns items along the main axis
<code>align-items</code>	Aligns items along the cross axis
<code>gap</code>	Sets spacing between items

Flex Direction

```
css
.container {
  display: flex;
  flex-direction: row;  /* Default: left to right */
}
```

Value	Description
<code>row</code>	Left to right (default)
<code>row-reverse</code>	Right to left

Value	Description
<code>column</code>	Top to bottom
<code>column-reverse</code>	Bottom to top

Flex Wrap

By default, flex items try to fit on one line. `flex-wrap: wrap` allows them to wrap to multiple lines.

```
css
.container {
  display: flex;
  flex-wrap: wrap;
}
```

Justify Content (Main Axis Alignment)

```
css
.container {
  display: flex;
  justify-content: center;
}
```

Value	Description
<code>flex-start</code>	Items align to the start (default)
<code>flex-end</code>	Items align to the end
<code>center</code>	Items are centered
<code>space-between</code>	Equal space between items
<code>space-around</code>	Equal space around items

Value	Description
space-evenly	Equal space distributed evenly

Align Items (Cross Axis Alignment)

```
css
.container {
  display: flex;
  align-items: center;
}
```

Value	Description
stretch	Items stretch to fill container (default)
flex-start	Items align to the start of the cross axis
flex-end	Items align to the end of the cross axis
center	Items are centered on the cross axis
baseline	Items align along their text baseline

Flex Item Properties

Property	Description
flex-grow	How much an item should grow relative to others
flex-shrink	How much an item should shrink relative to others
flex-basis	The initial size of an item before space is distributed

```
css
.item {
```

```
    flex-grow: 1; /* Items share available space equally */
}
```

Complete Flexbox Example

```
html
<!DOCTYPE html>
<html>
  <head>
    <title>Flexbox Layout</title>
    <style>
      .flex-container {
        display: flex;
        flex-wrap: wrap;
        justify-content: center;
        align-items: center;
        gap: 15px;
        background: #ecf0f1;
        padding: 20px;
        border: 2px solid #2c3e50;
        min-height: 200px;
      }
      .flex-item {
        background: #3498db;
        color: white;
        padding: 20px;
        border-radius: 5px;
        width: 150px;
        text-align: center;
      }
      .flex-item:nth-child(2) {
        background: #e74c3c;
      }
      .flex-item:nth-child(3) {
        background: #2ecc71;
      }
      .flex-item:nth-child(4) {
        background: #f39c12;
      }
    </style>
  </head>
  <body>
    <div class="flex-container">
      <div class="flex-item">Item 1</div>
      <div class="flex-item">Item 2</div>
      <div class="flex-item">Item 3</div>
      <div class="flex-item">Item 4</div>
    </div>
  </body>
</html>
```

```
    }
  </style>
</head>
<body>
  <h2>Flexbox Example</h2>
  <div class="flex-container">
    <div class="flex-item">Item 1</div>
    <div class="flex-item">Item 2</div>
    <div class="flex-item">Item 3</div>
    <div class="flex-item">Item 4</div>
  </div>
</body>
</html>
```

Part 2: CSS Media Queries

Introduction to Media Queries

Media queries allow you to apply different CSS styles based on the characteristics of the device displaying the page, such as screen width, height, or orientation. They are the foundation of responsive web design.

Media Query Syntax

```
css
@media media-type and (condition) {
  /* CSS rules */
}
```

Example:

```
css
@media screen and (max-width: 600px) {
  body {
    background-color: lightblue;
  }
}
```

```
}  
}
```

Common Media Query Breakpoints

Breakpoint	Target Device
<code>max-width: 600px</code>	Mobile phones
<code>max-width: 768px</code>	Tablets
<code>max-width: 1024px</code>	Small laptops
<code>min-width: 1025px</code>	Desktop

Simple Media Query Example

```
html  
<!DOCTYPE html>  
<html>  
  <head>  
    <title>Media Queries</title>  
    <style>  
      body {  
        font-family: Arial, sans-serif;  
        background: #f5f5f5;  
      }  
      .container {  
        display: flex;  
        gap: 20px;  
        padding: 20px;  
      }  
      .box {  
        background: #3498db;  
        color: white;  
        padding: 20px;  
        flex: 1;  
      }  
    </style>  
  </head>  
</html>
```

```
        text-align: center;
    }

    /* When screen is 768px or smaller */
    @media screen and (max-width: 768px) {
        .container {
            flex-direction: column;
        }
        .box {
            background: #e74c3c;
        }
    }

    /* When screen is 480px or smaller */
    @media screen and (max-width: 480px) {
        .box {
            background: #2ecc71;
            font-size: 14px;
            padding: 10px;
        }
    }
</style>
</head>
<body>
    <div class="container">
        <div class="box">Box 1</div>
        <div class="box">Box 2</div>
        <div class="box">Box 3</div>
    </div>
    <p>Resize your browser window to see the layout change.</p>
</body>
</html>
```

Part 3: CSS RWD – Responsive Web Development

Introduction to RWD

Responsive Web Design (RWD) is an approach to web design that makes web pages render well on a variety of devices and window or screen sizes. It uses a combination of flexible grids, flexible images, and media queries.

Key Principles of RWD

Principle	Description
Fluid Grids	Layouts use relative units (% , em, rem) instead of fixed pixels
Flexible Images	Images scale within their containers using <code>max-width: 100%</code>
Media Queries	Apply different styles based on screen characteristics

Fluid Grids

Use percentage-based widths instead of fixed pixel widths.

```
css
.column {
  width: 50%;
  float: left;
  box-sizing: border-box;
  padding: 10px;
}
```

Flexible Images

```
css
```

```
img {  
    max-width: 100%;  
    height: auto;  
}
```

Responsive Videos

```
css  
.video-container {  
    position: relative;  
    padding-bottom: 56.25%; /* 16:9 aspect ratio */  
    height: 0;  
    overflow: hidden;  
}  
.video-container iframe {  
    position: absolute;  
    top: 0;  
    left: 0;  
    width: 100%;  
    height: 100%;  
}
```

Complete RWD Example

```
<!DOCTYPE html>  
<html>  
    <head>  
        <title>Responsive Web Design</title>  
        <meta name="viewport" content="width=device-width, initial-scale=1.0">  
        <style>  
            * {  
                box-sizing: border-box;  
            }  
            body {  
                font-family: Arial, sans-serif;  
                margin: 0;  
                padding: 20px;  
                background: #f5f5f5;  
            }  
        </style>  
    </head>  
</html>
```

```
}

/* Grid Layout */
.row {
    display: flex;
    flex-wrap: wrap;
    gap: 20px;
}
.column {
    flex: 1;
    min-width: 200px;
    background: white;
    padding: 20px;
    border-radius: 10px;
    box-shadow: 0 2px 5px rgba(0,0,0,0.1);
}

/* Images */
img {
    max-width: 100%;
    height: auto;
    border-radius: 5px;
}

/* Navigation */
.nav {
    background: #2c3e50;
    padding: 10px;
    border-radius: 10px;
    margin-bottom: 20px;
}
.nav a {
    color: white;
    text-decoration: none;
    padding: 10px 15px;
    display: inline-block;
}
.nav a:hover {
    background: #3498db;
```

```

        border-radius: 5px;
    }

    /* Responsive: Tablets and smaller */
    @media screen and (max-width: 768px) {
        .column {
            flex: 1 1 100%;
        }
        .nav a {
            display: block;
            text-align: center;
            padding: 8px;
        }
    }

    /* Responsive: Mobile */
    @media screen and (max-width: 480px) {
        body {
            padding: 10px;
        }
        .column {
            padding: 15px;
        }
        h1 {
            font-size: 24px;
        }
    }
</style>
</head>
<body>
    <h1>Responsive Website</h1>

    <div class="nav">
        <a href="#">Home</a>
        <a href="#">About</a>
        <a href="#">Services</a>
        <a href="#">Contact</a>
    </div>

```

```
<div class="row">
  <div class="column">
    <h2>Column 1</h2>
    
    <p>This column adapts to screen size using flexbox and media queries.
  </p>
</div>
<div class="column">
  <h2>Column 2</h2>
  
  <p>Resize your browser to see the responsive behavior in action.</p>
</div>
</div>
</body>
</html>
```

Part 4: Introduction to JavaScript

What Is JavaScript?

JavaScript is a programming language that makes web pages interactive. While HTML provides structure and CSS provides styling, JavaScript provides behavior — it allows web pages to respond to user actions, manipulate content dynamically, and communicate with servers.

Key Facts:

- Developed by Brendan Eich at Netscape in 1995
- Originally called Mocha, then LiveScript, then JavaScript
- Not related to Java (the name was a marketing decision)
- Runs in the browser (client-side) and on servers (Node.js)

JavaScript Syntax Basics

Comments:

```
javascript
// This is a single-line comment

/*
This is a
multi-line comment
*/
```

Case Sensitivity: JavaScript is case-sensitive. `myVariable` and `myvariable` are different.

Semicolons: Statements typically end with a semicolon (;), though they are sometimes optional.

Hello World Example

```
javascript
// Display a message in an alert box
alert("Hello, World!");

// Display a message in the browser console
console.log("Hello, World!");

// Display a message on the web page
document.write("Hello, World!");
```

Including JavaScript in HTML

Inline JavaScript:

```
html
<button onclick="alert('Hello!')">Click Me</button>
```

Internal JavaScript (in the `<head>` or before `</body>`):

```
<!DOCTYPE html>
<html>
```

```
<head>
  <title>JavaScript Example</title>
</head>
<body>
  <h1>JavaScript Demo</h1>
  <script>
    console.log("This runs in the browser console.");
  </script>
</body>
</html>
```

External JavaScript (in a separate `.js` file):

```
html
<!DOCTYPE html>
<html>
  <head>
    <title>JavaScript Example</title>
  </head>
  <body>
    <script src="script.js"></script>
  </body>
</html>

javascript
// script.js
console.log("This is from an external file.");
```

Part 5: Algorithms, Flowcharts, and Pseudocode

What Is an Algorithm?

An **algorithm** is a step-by-step procedure or set of rules for solving a problem or performing a task. It is the foundation of all programming.

Example Algorithm: Making a Cup of Coffee

1. Fill the kettle with water
2. Turn on the kettle
3. Put coffee grounds in a cup
4. Pour hot water into the cup
5. Stir the coffee
6. Add milk and sugar (optional)
7. Drink the coffee

Algorithm Characteristics

Characteristic	Description
Finite	Must terminate after a finite number of steps
Definite	Each step must be precisely defined
Input	Must have zero or more inputs
Output	Must produce at least one output
Effective	Each step must be executable

Flowcharts

A **flowchart** is a visual representation of an algorithm using symbols and arrows to show the sequence of steps.

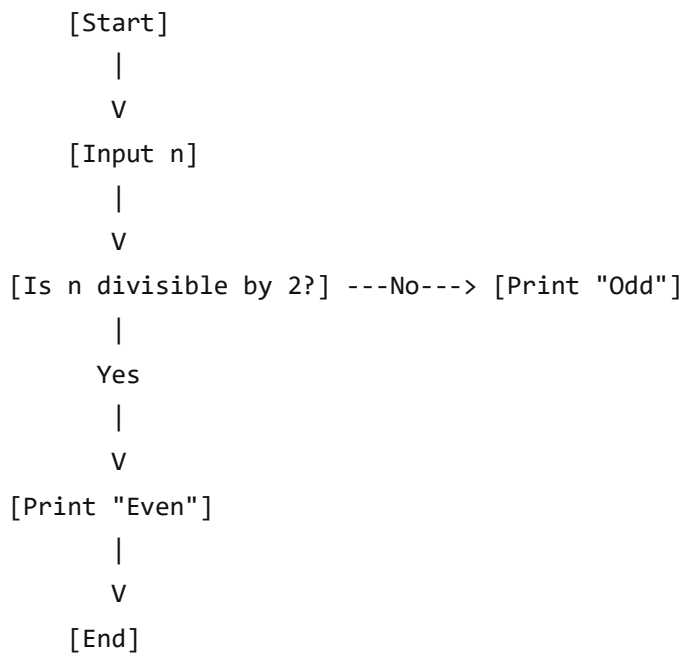
Common Flowchart Symbols:

Symbol	Name	Meaning
Oval	Start/End	Indicates the beginning or end of the algorithm
Rectangle	Process/Assignment	Indicates a step or action

Symbol	Name	Meaning
Parallelogram	Input/Output	Indicates input or output
Diamond	Decision	Indicates a condition that branches (yes/no)
Arrow	Flow Line	Shows the direction of the process

Flowchart Example: Check if a Number is Even or Odd

text



Pseudocode

Pseudocode is a simplified, informal description of an algorithm written in plain language mixed with programming-like syntax. It is not actual code but a way to plan logic before coding.

Pseudocode Example: Even or Odd Checker

```
text
BEGIN
    INPUT number
    IF number MOD 2 == 0 THEN
        PRINT "Even"
    ELSE
        PRINT "Odd"
    ENDIF
END
```

Flowchart Example (Text Representation)

```

text
[Start] → [Input number] → [Is number < 0?]
      |
      | Yes      |      No
      |          |          |
      V          |          V
[Print "Negative"] | [Is number > 0?]
      |          |          |
      |          | Yes    |      No
      |          |      |      |
      |          |      V      V
      | [Print "Positive"] [Print "Zero"]
      |          |          |
      +-----+-----+-----+
      |
      V
[End]

```

Part 6: Variables, Data Types, and Functions

Variables

A **variable** is a container for storing data values. In JavaScript, variables are declared using `var`, `let`, or `const`.

Keyword	Description
<code>var</code>	The old way (function-scoped, avoid in modern code)
<code>let</code>	The modern way (block-scoped, can be reassigned)
<code>const</code>	Block-scoped, cannot be reassigned (value is constant)

javascript

// Using let

`let name = "Juan";`

`let age = 17;`

`let isStudent = true;`

// Using const

`const PI = 3.14159;`

`const birthYear = 2007;`

// Changing a let variable

`name = "Maria";` *// Works*

// Changing a const variable

// PI = 3.14; // Error! Cannot reassign a constant

Data Types

JavaScript has several data types. The main ones are:

Data Type	Description	Example
<code>number</code>	Numeric values (integers and decimals)	<code>42, 3.14</code>
<code>string</code>	Text values (in quotes)	<code>"Hello", 'World'</code>
<code>boolean</code>	True or false values	<code>true, false</code>
<code>undefined</code>	Variable declared but no value assigned	<code>let x;</code>
<code>null</code>	Intentionally empty value	<code>let y = null;</code>
<code>object</code>	Collections of key-value pairs	<code>{name: "Juan", age: 17}</code>
<code>array</code>	Ordered lists of values	<code>[1, 2, 3, 4]</code>

javascript

// Numbers

```
let age = 17;
```

```
let price = 99.99;
```

// Strings

```
let name = "Juan Dela Cruz";
```

```
let greeting = 'Hello, World!';
```

// Booleans

```
let isLoggedIn = true;
```

```
let isAdmin = false;
```

// Arrays

```
let colors = ["red", "green", "blue"];
```

```
let scores = [98, 87, 92];
```

```
// Objects
let student = {
  name: "Juan",
  age: 17,
  grade: 11
};
```

Functions

A **function** is a block of code designed to perform a particular task. Functions allow you to reuse code, organize logic, and make programs more readable.

Function Syntax:

```
javascript
function functionName(parameters) {
  // Code to execute
  return value; // Optional
}
```

Simple Function:

```
javascript
function greet(name) {
  return "Hello, " + name + "!";
}

// Call the function
let message = greet("Juan");
console.log(message); // Output: Hello, Juan!
```

Function with Multiple Parameters:

```
javascript
function add(a, b) {
  return a + b;
}

let sum = add(5, 3);
console.log(sum); // Output: 8
```

Common JavaScript Operations

```
javascript
// Arithmetic Operations
let a = 10;
let b = 3;
let sum = a + b;      // 13
let difference = a - b; // 7
let product = a * b;   // 30
let quotient = a / b;  // 3.333...
let remainder = a % b; // 1

// Comparison Operators
let x = 5;
let y = 10;
let isEqual = x == y;    // false
let isNotEqual = x != y; // true
let isGreater = x > y;   // false
let isLess = x < y;      // true

// String Concatenation
let firstName = "Juan";
let lastName = "DeLa Cruz";
let fullName = firstName + " " + lastName; // "Juan DeLa Cruz"

// Template Literals
let age = 17;
let intro = `My name is ${firstName} and I am ${age} years old.`;
```

Part 7: Complete Web Page with JavaScript

```
html
<!DOCTYPE html>
<html>
  <head>
    <title>Interactive Web Page</title>
    <style>
      * {
        box-sizing: border-box;
      }
      body {
        font-family: Arial, sans-serif;
        background: #f5f5f5;
        padding: 20px;
      }
      .container {
        max-width: 800px;
        margin: 0 auto;
        background: white;
        padding: 30px;
        border-radius: 10px;
        box-shadow: 0 4px 8px rgba(0,0,0,0.1);
      }
      .flex-row {
        display: flex;
        flex-wrap: wrap;
        gap: 15px;
        margin: 20px 0;
      }
      .flex-row input {
        flex: 1;
        padding: 10px;
        border: 1px solid #ccc;
        border-radius: 5px;
        min-width: 150px;
```

```

    }
    .flex-row button {
        padding: 10px 20px;
        background: #3498db;
        color: white;
        border: none;
        border-radius: 5px;
        cursor: pointer;
    }
    .flex-row button:hover {
        background: #2980b9;
    }
    .result {
        background: #ecf0f1;
        padding: 15px;
        border-radius: 5px;
        margin: 15px 0;
    }
}

@media screen and (max-width: 500px) {
    .flex-row {
        flex-direction: column;
    }
    .flex-row input,
    .flex-row button {
        width: 100%;
    }
}
</style>
</head>
<body>
    <div class="container">
        <h1>Interactive Calculator</h1>

        <div class="flex-row">
            <input type="number" id="num1" placeholder="Enter first number">
            <input type="number" id="num2" placeholder="Enter second number">
            <button onclick="addNumbers()">Add</button>
            <button onclick="multiplyNumbers()">Multiply</button>

```



```

        <button onclick="clearResult()">Clear</button>
    </div>

    <div class="result" id="resultDisplay">
        <p>Result will appear here.</p>
    </div>

    <h2>Simple Greeting</h2>
    <div class="flex-row">
        <input type="text" id="nameInput" placeholder="Enter your name">
        <button onclick="greetUser()">Greet</button>
    </div>
    <div class="result" id="greetingDisplay">
        <p>Enter your name and click Greet.</p>
    </div>
</div>

<script>
    // Function to add two numbers
    function addNumbers() {
        let num1 = parseFloat(document.getElementById('num1').value);
        let num2 = parseFloat(document.getElementById('num2').value);

        if (isNaN(num1) || isNaN(num2)) {
            document.getElementById('resultDisplay').innerHTML = '<p style="color: red;">Please enter valid numbers.</p>';
            return;
        }

        let sum = num1 + num2;
        document.getElementById('resultDisplay').innerHTML = `<p>${num1} + ${num2} = <strong>${sum}</strong></p>`;
    }

    // Function to multiply two numbers
    function multiplyNumbers() {
        let num1 = parseFloat(document.getElementById('num1').value);
        let num2 = parseFloat(document.getElementById('num2').value);

```

```

        if (isNaN(num1) || isNaN(num2)) {
            document.getElementById('resultDisplay').innerHTML = '<p style="color: red;">Please enter valid numbers.</p>';
            return;
        }

        let product = num1 * num2;
        document.getElementById('resultDisplay').innerHTML = `<p>${num1} × ${num2} = <strong>${product}</strong></p>`;
    }

    // Function to greet user
    function greetUser() {
        let name = document.getElementById('nameInput').value.trim();

        if (name === "") {
            document.getElementById('greetingDisplay').innerHTML = '<p style="color: red;">Please enter your name.</p>';
            return;
        }

        let greeting = `Hello, ${name}! Welcome to our interactive page.`;
        document.getElementById('greetingDisplay').innerHTML = `<p>${greeting}</p>`;
    }

    // Function to clear result
    function clearResult() {
        document.getElementById('resultDisplay').innerHTML = '<p>Result will appear here.</p>';
        document.getElementById('num1').value = '';
        document.getElementById('num2').value = '';
    }

    // Console message
    console.log("Interactive page loaded successfully!");
</script>
</body>
</html>

```

Key Terms

Term	Definition
Flexbox	CSS layout model for one-dimensional layouts (rows or columns)
Media Query	CSS technique to apply styles based on screen characteristics
Responsive Web Design	Approach to making websites work on all device sizes
Breakpoint	A specific screen width where a design changes
RWD	Responsive Web Design
JavaScript	A programming language that adds interactivity to web pages
Algorithm	A step-by-step procedure for solving a problem
Flowchart	A visual representation of an algorithm using symbols
Pseudocode	An informal description of an algorithm in plain language
Variable	A container for storing data values
Data Type	The kind of data a variable can hold (number, string, boolean, etc.)
Function	A reusable block of code that performs a specific task
Console	A tool in browsers for debugging and testing code

Review Questions

Write your answers on a separate sheet of paper.

CSS Flexbox and RWD

1. What is the purpose of CSS Flexbox? Name three flexbox properties and describe what each does.
2. Write a CSS rule that creates a flex container with items centered both horizontally and vertically.
3. What is a media query and why is it important in responsive web design?
4. Write a media query that changes the background color to light gray on screens smaller than 600px.
5. What is the difference between `flex-direction: row` and `flex-direction: column`?
6. Explain the concept of responsive web design and list its three key principles.
7. Write a CSS rule to make images responsive so they do not overflow their containers.

JavaScript Fundamentals

8. What is JavaScript and what is its primary purpose in web development?
9. Write JavaScript code that prints "Hello, World!" to the browser console.
10. Explain the difference between `let` and `const` in JavaScript. When would you use each?
11. Identify the data type of each of the following values:
 - `"Hello"`
 - `42`
 - `true`
 - `[1, 2, 3]`
 - `null`
 - `{ name: "Juan" }`

12. Write a JavaScript function named `multiply` that takes two parameters and returns their product.

Algorithms, Flowcharts, and Pseudocode

13. What is an algorithm? Give one example of an algorithm from daily life.

14. Draw a flowchart (or describe in text) for checking whether a number is positive, negative, or zero.

15. Write pseudocode for an algorithm that determines the larger of two numbers.

16. Explain the purpose of pseudocode in programming.

Mixed Problems

17. Create a complete HTML page that uses CSS Flexbox to display three boxes in a row on desktop but stacked vertically on mobile devices.

18. Write a JavaScript program that asks the user for their age and displays whether they are a minor (under 18) or an adult (18 and above).

Answer Key

1. Flexbox is a CSS layout model for arranging items in rows or columns.

- `display: flex` – defines a flex container
- `flex-direction` – sets the direction (row, column)
- `justify-content` – aligns items on the main axis
- `align-items` – aligns items on the cross axis
- `flex-wrap` – controls whether items wrap

2.

css

```
.container {  
  display: flex;  
  justify-content: center;  
  align-items: center;
```

```
height: 100vh; }
```

3. A media query applies CSS styles only when certain conditions are met (e.g., screen width). It is essential for responsive design because it allows layouts to adapt to different devices.

4.

```
css
@media screen and (max-width: 600px) {
  body {
    background-color: lightgray;
  }
}
```

5. `flex-direction: row` arranges items horizontally (left to right). `flex-direction: column` arranges items vertically (top to bottom).

6. Responsive web design is an approach that makes web pages work on all devices. Three key principles are:

- Fluid grids (relative units instead of pixels)
- Flexible images (max-width: 100%)
- Media queries (different styles for different screen sizes)

7.

```
css
img {
  max-width: 100%;
  height: auto;
}
```

8. JavaScript is a programming language that adds interactivity to web pages. It allows dynamic content, user interactions, and server communication.

9.

```
javascript
console.log("Hello, World!");
```

10. `let` is used for variables that can be reassigned. `const` is used for variables that should not be reassigned. Use `const` by default, use `let` when you need to change the value.

11.

- `"Hello"` → string
- `42` → number
- `true` → boolean
- `[1, 2, 3]` → array (object)
- `null` → null (object)
- `{ name: "Juan" }` → object

12.

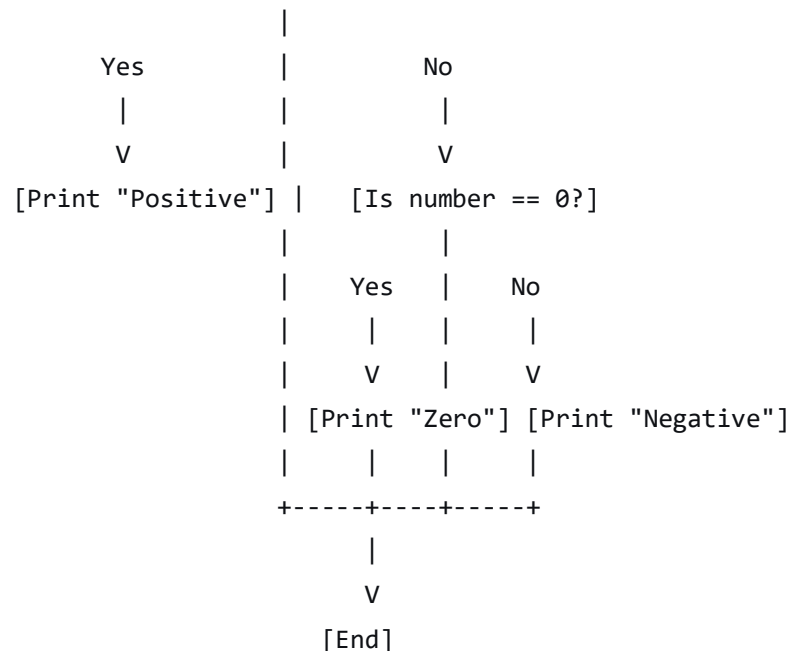
```
javascript
function multiply(a, b) {
    return a * b; }
```

13. An algorithm is a step-by-step procedure for solving a problem. Example: Recipe for cooking a meal, directions to a location.

14. Flowchart (text description):

text

```
[Start] → [Input number] → [Is number > 0?]
```



15. Pseudocode:

```
text
BEGIN
    INPUT number1, number2
    IF number1 > number2 THEN
        PRINT number1 + " is larger"
    ELSE IF number2 > number1 THEN
        PRINT number2 + " is larger"
    ELSE
        PRINT "Both numbers are equal"
    ENDIF
END
```

16. Pseudocode helps programmers plan the logic of a program before writing actual code. It allows thinking through the steps without worrying about syntax.

17.

```
html
<!DOCTYPE html>
<html>
    <head>
        <title>Responsive Layout</title>
        <style>
            * { box-sizing: border-box; }
            .container {
                display: flex;
                gap: 20px;
                padding: 20px;
            }
            .box {
                flex: 1;
                background: #3498db;
                color: white;
                padding: 40px;
                text-align: center;
                border-radius: 10px;
            }
            @media screen and (max-width: 600px) {
```



```
        .container {
            flex-direction: column;
        }
    }
</style>
</head>
<body>
    <div class="container">
        <div class="box">Box 1</div>
        <div class="box">Box 2</div>
        <div class="box">Box 3</div>
    </div>
</body>
</html>
```

18.

```
javascript
let age = prompt("Enter your age:");
if (age < 18) {
    console.log("You are a minor.");
} else {
    console.log("You are an adult.");
}
```